

Email Parser Lab (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p3-email-parser-lab>  <https://github.com/learn-co-curriculum/python-p3-email-parser-lab/issues/new>

Learning Goals

- Validate that strings match specific patterns using regular expressions.
 - Search for strings that match specific patterns using regular expressions.
-

Key Vocab

- **Class**: a bundle of data and functionality. Can be copied and modified to accomplish a wide variety of programming tasks.
 - **Object**: the more common name for an instance. The two can usually be used interchangeably.
 - **Object-Oriented Programming**: programming that is oriented around data (made mobile and changeable in **objects**) rather than functionality. Python is an object-oriented programming language.
 - **Function**: a series of steps that create, transform, and move data.
 - **Method**: a function that is defined inside of a class.
-

Instructions

This is a **test-driven lab**. Run `pipenv install` to create your virtual environment and `pipenv shell` to enter the virtual environment. Then run `pytest -x` to run your tests. Use these instructions and `pytest` 's error messages to complete your work in the `lib/` folder.

You will write a program that, given a string of email addresses, parses that string into an array.

Your class, `EmailAddressParser`, should take the string of addresses on initialization. Instances should respond to a `parse()` instance method that parses the string into individual email addresses and returns them in an array.

Your class should be able to do this:

```
email_addresses = "john@doe.com, person@somewhere.org"
parser = EmailAddressParser(email_addresses)

parser.parse()
# => ["john@doe.com", "person@somewhere.org"]
```

Note: Your `EmailAddressParser` class should handle a list of email addresses that are separated by either spaces *or* comma-separated values (CSVs):

```
EmailAddressParser("john@doe.com, person@somewhere.org").parse
# => ["john@doe.com", "person@somewhere.org"]

EmailAddressParser("john@doe.com person@somewhere.org").parse
# => ["john@doe.com", "person@somewhere.org"]
```

Additionally, the `parse()` method should *only return **unique** addresses*, sorted alphabetically.

Write your solution in `lib/email_address_parser.py`.

Once all of your tests are passing, commit and push your work using `git` to submit.

Hints: in order to get this lab passing, you will need to do some research into the following topics:

- [re - Regular expression operations - Python](https://docs.python.org/3/library/re.html)  [\(https://docs.python.org/3/library/re.html\)](https://docs.python.org/3/library/re.html)
- [Python Regex Split String Using re.split\(\)](https://pynative.com/python-regex-split/)  [\(https://pynative.com/python-regex-split/\)](https://pynative.com/python-regex-split/)
- Think back to data structures: what could we cast a list to in order to get unique values?

Resources

- [re - Regular expression operations - Python](https://docs.python.org/3/library/re.html) ↗ (https://docs.python.org/3/library/re.html)
- [Python Regex Split String Using re.split\(\)](https://pynative.com/python-regex-split/) ↗ (https://pynative.com/python-regex-split/)

This tool needs to be loaded in a new browser window

Load Email Parser Lab (CodeGrade) in a new window